

MODULE 7

Auditer, prouver, rendre

Module 7. Auditer, prouver, rendre

Module de clôture : scan dynamique automatisé de TaskFlow, audit des dépendances, écriture de tests de sécurité Jest, et rédaction du rapport de sécurisation professionnel.

Objectifs opérationnels

À la fin de ce module, tu auras dans ton application : - ☒ Scan OWASP ZAP baseline exécuté et résultats analysés - ☒ `npm audit` passé avec zéro vulnérabilité critique - ☒ 5 tests Jest de sécurité écrits et passants - ☒ Rapport final `docs/rapport-final.md` (ou PDF) rédigé - ☒ Checklist OWASP Top 10 complétée - ☒ Code taggé `M7-audit-final`

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
Rappels	Concepts 1-10	Lecture active	30 min
1	OWASP ZAP	Scan baseline	20 min
2	npm audit	Remédiation dépendances	15 min
3	Tests Jest	5 tests de sécurité	25 min
4	Rapport	Template complet	20 min
5	Checklist OWASP	Cocher les items	5 min
6	Risques résiduels	Documenter ce qui n'est pas (encore) traité	5 min

CONCEPT 1, DAST (Dynamic Application Security Testing)

Rappel. Le DAST teste l'application en cours d'exécution, depuis l'extérieur, comme le ferait un attaquant. Contrairement au SAST (analyse statique du code), le DAST découvre les vulnérabilités qui n'apparaissent qu'au runtime : injections, mauvaise configuration des headers HTTP, endpoints non protégés, redirections ouvertes. Il ne nécessite pas d'accès au code source. Les outils DAST envoient des milliers de payloads

automatiquement et analysent les réponses pour détecter des comportements anormaux. OWASP ZAP est l'outil DAST open source de référence.

Risque concret	Des vulnérabilités invisibles à la lecture du code (ex. : header manquant, endpoint non listé dans la doc) restent non détectées sans scan dynamique.
Exemple de faille	Route <code>/admin/debug</code> non documentée, accessible sans authentification, le SAST ne la trouve pas, le DAST si.
À vérifier dans l'app	Lancer l'application sur HTTPS port 3443 avant le scan
Correction	Corriger les findings ZAP et relancer le scan
Test de validation	Scan ZAP baseline : 0 alerte HIGH ou CRITICAL après corrections
Trace rapport	« Un scan DAST baseline a été exécuté avec OWASP ZAP. X alertes ont été identifiées (Y HIGH, Z MEDIUM). Toutes les alertes HIGH ont été corrigées. »

CONCEPT 2, OWASP ZAP, Scan Automatique

Rappel. ZAP (Zed Attack Proxy) est un proxy HTTP qui intercepte le trafic entre le navigateur et l'application, et lance des attaques automatisées. Le mode "Baseline Scan" lance une araignée passive et des tests actifs limités, adapté à une CI/CD. Le mode "Full Scan" est plus agressif. ZAP classe les alertes par sévérité : High, Medium, Low, Informational. Les faux positifs sont fréquents, chaque alerte doit être vérifiée manuellement avant d'être incluse dans le rapport. ZAP est disponible en version Desktop (GUI) et en ligne de commande via Docker.

Risque concret	Sans scan automatisé, des classes entières de vulnérabilités (headers manquants, XSS reflété, CSRF) peuvent passer inaperçues lors d'une revue manuelle.
Exemple de faille	Header <code>X-Content-Type-Options</code> absent, ZAP le détecte automatiquement, une revue de code ne le voit pas.
À vérifier dans l'app	Application démarrée sur <code>https://localhost:3443</code> avant le scan
Correction	Appliquer les corrections Helmet (M5) avant le scan pour réduire le bruit
Test de validation	Rapport HTML ZAP généré dans <code>proofs/M7/zap-report.html</code>
Trace rapport	« Scan ZAP baseline effectué le [DATE]. Rapport complet dans <code>proofs/M7/zap-report.html</code> . Résumé : X High, Y Medium, Z Low, W Informational. »

Installation et scan via Docker :

```
# Démarrer l'application
cd vuln-app && node app.js &

# Scan ZAP baseline via Docker (recommandé, pas d'installation locale)
docker pull ghcr.io/zaproxy/zaproxy:stable

docker run --rm \
  --network host \
  -v $(pwd)/proofs/M7:/zap/wrk:rw \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py \
  -t https://localhost:3443 \
  -r zap-report.html \
  -I \
  --hook=/zap/auth_hook.py 2>/dev/null || true

# Le rapport sera dans proofs/M7/zap-report.html
```

Alternative, ZAP Desktop (Windows/Mac) :

1. Télécharger sur <https://www.zaproxy.org/download/>
2. Démarrer ZAP
3. Quick Start → Automated Scan
4. URL : <https://localhost:3443>
5. Accepter le certificat ZAP dans le navigateur
6. Launch Browser → Explorer l'application manuellement
7. Active Scan → Démarrer
8. Report → Generate HTML Report → sauver dans proofs/M7/

TP, Étape 1 : Scan ZAP

```
mkdir -p proofs/M7

# Option 1 : Docker
docker run --rm --network host \
  -v "$(pwd)/proofs/M7:/zap/wrk" \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t https://localhost:3443 -r zap-report.html -I

# Option 2 : ZAP Desktop (interface graphique)
# Voir instructions ci-dessus

echo "Rapport généré dans proofs/M7/zap-report.html"
```

✓ **Point de vérification** : le fichier `proofs/M7/zap-report.html` existe et contient des résultats.

CONCEPT 3, Interprétation des résultats ZAP

Rappel. ZAP génère des alertes classées par sévérité. Un faux positif est une alerte qui décrit une vulnérabilité qui n'existe pas réellement dans le contexte de l'application. Il faut analyser chaque alerte : lire la description, reproduire manuellement si possible, et décider de l'accepter, la corriger, ou la marquer comme faux positif documenté. Les alertes "Informational" sont généralement des observations (ex. : cookies sans expiration) qui peuvent ne pas nécessiter de correction immédiate.

Risque concret	Ignorer toutes les alertes ZAP parce qu'elles semblent trop nombreuses → passer à côté d'une vulnérabilité réelle HIGH.
Exemple de faille	Alerte "SQL Injection" détectée sur <code>/tasks</code> → à reproduire avec <code>sqlmap</code> pour confirmer
À vérifier dans l'app	Trier les alertes ZAP par sévérité, traiter en priorité HIGH puis MEDIUM
Correction	Pour chaque alerte HIGH : reproduire, corriger, retester
Test de validation	Relancer le scan ZAP après corrections → 0 alerte HIGH
Trace rapport	« Sur X alertes ZAP, Y ont été qualifiées de faux positifs (documentés), Z ont conduit à des corrections (listées section 4 du rapport). »

Tableau de triage des alertes :

ALERTE ZAP	SÉVÉRITÉ	FAUX POSITIF ?	ACTION
Missing Anti-CSRF Tokens	High	Non (si routes HTML)	Corriger, M5
X-Frame-Options Header Not Set	Medium	Non	Corriger, Helmet
Content Security Policy Not Set	Medium	Non	Corriger, M5
Application Error Disclosure	Medium	Vérifier	Corriger, M6
Cookie No HttpOnly Flag	Low	Non	Corriger, M5
Server Leaks Version Information	Low	Oui si Helmet actif	Documenter

CONCEPT 4, SCA (Software Composition Analysis)

Rappel. Une application Node.js typique dépend de dizaines de bibliothèques tierces. Chacune peut contenir des vulnérabilités connues, répertoriées dans des bases publiques comme le NVD (National Vulnerability Database) ou Snyk. `npm audit` interroge automatiquement le registre npm pour identifier les dépendances vulnérables et suggère des mises à jour. `npm audit fix` applique les corrections compatibles (patch, minor). `npm audit fix --force` applique les corrections avec changements majeurs, à faire avec précaution car cela peut casser l'API.

Risque concret	<code>express@4.17.1</code> avait une vulnérabilité de déni de service. Une dépendance non mise à jour est un vecteur d'attaque bien documenté (OWASP A06).
Exemple de faille	<code>package.json</code> avec versions non pinnées : <code>"express": "^4.0.0"</code> → versions vulnérables acceptées
À vérifier dans l'app	<code>npm audit</code> dans le dossier <code>vuln-app/</code>
Correction	<code>npm audit fix</code> puis vérifier que les tests passent
Test de validation	<code>npm audit</code> → 0 vulnerability de sévérité high ou critical
Trace rapport	« <code>npm audit</code> a révélé X vulnérabilités (Y critical, Z high). Après <code>npm audit fix</code> , X vulnérabilités subsistent (risques résiduels documentés en section 6). »

TP, Étape 2 : npm audit

```
cd vuln-app

# Audit initial
npm audit

# Sauvegarder le rapport initial
npm audit --json > ../proofs/M7/npm-audit-before.json

# Corriger les vulnérabilités automatiquement
npm audit fix

# Vérifier ce qui reste
npm audit

# Sauvegarder le rapport final
npm audit --json > ../proofs/M7/npm-audit-after.json

# Corrections manuelles si nécessaire
npm audit fix --force # attention : peut casser des APIs
```

✓ **Point de vérification :** `npm audit` retourne 0 vulnérabilités HIGH ou CRITICAL.

CONCEPT 5, Pinning des versions

Rappel. `package.json` utilise des plages de versions (`^1.2.3` , `~1.2.3` , `*`) qui permettent à `npm install` de télécharger automatiquement des versions plus récentes. `package-lock.json` fixe les versions exactes de toutes les dépendances transitives. Il doit être commité dans le dépôt pour garantir des builds reproductibles. Sur un projet en production, éviter `^` (accepte les mises à jour minor) et préférer des versions fixes pour les dépendances critiques de sécurité (express, jsonwebtoken, bcrypt).

Risque concret	<code>^4.0.0</code> accepte <code>4.18.1</code> qui pourrait introduire une régression de sécurité. Sans <code>package-lock.json</code> , deux développeurs peuvent avoir des versions différentes.
Exemple de faille	<code>package-lock.json</code> absent du dépôt ou dans <code>.gitignore</code> → builds non reproductibles
À vérifier dans l'app	<code>cat vuln-app/.gitignore \</code> <code>grep package-lock</code> → ne doit pas apparaître
Correction	<code>git add package-lock.json && git commit -m "pin: lock dependencies"</code>
Test de validation	<code>git log --oneline -- package-lock.json</code> → au moins un commit
Trace rapport	« <code>package-lock.json</code> est versionné. Les versions critiques (express, jsonwebtoken, bcrypt) ont été fixées sans préfixe <code>^</code> . »

```
# Vérifier que package-lock.json est commité
git log --oneline -- package-lock.json

# Si absent : commiter
git add package-lock.json
git commit -m "pin: add package-lock.json"
```

CONCEPT 6, Tests Automatisés de Sécurité (Jest)

Rappel. Les tests de sécurité automatisés vérifient que les corrections restent en place à chaque déploiement. Ils jouent les rôles d'attaquant : tenter une injection SQL, un IDOR, un XSS. Si le test passe (l'attaque échoue), la correction est en place. S'il échoue (l'attaque réussit), une régression a été introduite. Ces tests s'intègrent dans la CI/CD comme des tests unitaires classiques. `supertest` permet de tester les routes Express sans démarrer un vrai serveur HTTP.

Risque concret	Une correction SQL appliquée en M2 peut être accidentellement annulée lors d'un refactor → sans test automatisé, la vulnérabilité revient en production.
Exemple de faille	Absence totale de tests de sécurité dans <code>vuln-app/tests/</code>
À vérifier dans l'app	<code>ls vuln-app/tests/security/</code> → dossier absent
Correction	Écrire 5 tests Jest couvrant les failles principales
Test de validation	<code>npm test -- tests/security/</code> → 5 tests passants
Trace rapport	« 5 tests de sécurité automatisés ont été écrits. Ils couvrent : échec de login, IDOR, SQLi, XSS, headers. Tous passent en CI. »

Installation :

```
npm install --save-dev jest supertest
```

Ajouter dans `package.json` :

```
{
  "scripts": {
    "test": "jest",
    "test:security": "jest tests/security/"
  },
  "jest": {
    "testEnvironment": "node"
  }
}
```

Les 5 tests de sécurité :

```
// vuln-app/tests/security/security.test.js
const request = require('supertest');
const app = require('../..../app');

// -----
// TEST 1, Échec de login avec mauvais credentials
// -----
describe('Auth, Login sécurisé', () => {
  test('Login avec mauvais mot de passe retourne 401', async () => {
    const res = await request(app)
      .post('/login')
      .send({ email: 'test@test.com', password: 'wrong_password' });

    expect(res.statusCode).toBe(401);
    expect(res.body).not.toHaveProperty('token');
    expect(res.body.error).toBeDefined();
    // Vérifier que le message ne révèle pas si l'email existe
    expect(res.body.error).not.toMatch(/email.*(exist|trouvé)/i);
  });

  test('Réponse de login ne contient pas le mot de passe', async () => {
    const res = await request(app)
      .post('/login')
      .send({ email: 'test@test.com', password: 'any' });

    expect(JSON.stringify(res.body)).not.toMatch(/password/i);
  });
});

// -----
// TEST 2, IDOR : accès aux tâches d'un autre utilisateur bloqué
// -----
describe('Autorisation, IDOR bloqué', () => {
  test('Accéder à une tâche sans être connecté retourne 401', async () => {
    const res = await request(app).get('/tasks/1');
    expect(res.statusCode).toBe(401);
  });

  test('Un utilisateur ne peut pas voir les tâches d\'un autre (simulation)', async () => {
    // Simuler une session avec userId=2 tentant d'accéder à une tâche de userId=1
    const agent = request.agent(app);

    // Login avec user 2 (si existant dans la DB de test)
    // Ce test documente le comportement attendu
    const res = await agent.get('/tasks/999'); // ID inexistant pour un autre user
    expect([401, 403, 404]).toContain(res.statusCode);
  });
});

// -----
```

```

// TEST 3, Injection SQL bloquée
// -----
describe('Injection SQL, bloquée', () => {
  test('Payload SQLi dans login ne retourne pas de données utilisateur', async ()
=> {
    const sqliPayloads = [
      "' OR '1'='1",
      "' OR 1=1--",
      "admin'--",
      "' UNION SELECT * FROM users--",
    ];

    for (const payload of sqliPayloads) {
      const res = await request(app)
        .post('/login')
        .send({ email: payload, password: payload });

      // L'attaque ne doit pas aboutir à un login réussi
      expect(res.statusCode).not.toBe(200);
      expect(res.body).not.toHaveProperty('userId');
    }
  });
});

// -----
// TEST 4, XSS : payloads stockés ou reflétés bloqués
// -----
describe('XSS, payloads bloqués', () => {
  test('Payload XSS dans création de tâche est sanitisé ou refusé', async () => {
    // Note : ce test nécessite une session valide
    // On vérifie que le payload n'est pas retourné tel quel
    const xssPayload = '<script>alert("xss")</script>';

    const res = await request(app)
      .post('/tasks')
      .send({ title: xssPayload, description: xssPayload });

    // Soit la requête est refusée, soit le payload est échappé
    if (res.statusCode === 200 || res.statusCode === 201) {
      // Si accepté, vérifier que le script n'est pas retourné brut
      expect(res.text).not.toContain('<script>alert("xss")</script>');
    } else {
      // Refus explicite du payload
      expect([400, 401, 403, 422]).toContain(res.statusCode);
    }
  });
});

// -----
// TEST 5, Headers de sécurité présents
// -----
describe('Headers de sécurité, Helmet actif', () => {

```

```

test('Headers critiques présents sur la page d\'accueil', async () => {
  const res = await request(app).get('/');

  // Content Security Policy
  expect(res.headers).toHaveProperty('content-security-policy');

  // X-Frame-Options
  expect(res.headers['x-frame-options']).toBeDefined();
  expect(res.headers['x-frame-
options'].toUpperCase()).toMatch(/DENY|SAMEORIGIN/);

  // X-Content-Type-Options
  expect(res.headers['x-content-type-options']).toBe('nosniff');
});

test('Header Server ne révèle pas la version Express', async () => {
  const res = await request(app).get('/');
  // Helmet supprime ou masque le header Server
  expect(res.headers['x-powered-by']).toBeUndefined();
});
});

```

TP, Étape 3 : Écrire et exécuter les tests

```

cd vuln-app
mkdir -p tests/security
# Copier le fichier de tests ci-dessus dans tests/security/security.test.js

npm install --save-dev jest supertest

# Lancer les tests
npm test -- tests/security/security.test.js --verbose

# Sauvegarder le résultat
npm test -- tests/security/ --verbose > ../proofs/M7/jest-results.txt 2>&1

```

✓ **Point de vérification :** `npm test` → 5 tests, 5 passed .

CONCEPT 7, Audit Log Applicatif

Rappel. L'audit log enregistre les actions sensibles avec le contexte complet : qui (userId, IP), quoi (action), quand (timestamp ISO), résultat (succès/échec). Il permet la détection d'intrusion a posteriori et est souvent requis par les réglementations (RGPD, SOC2, PCI-DSS). Les événements à logger impérativement : authentification (réussi/

échec), modification de profil, accès admin, suppression de données, changement de permissions. Le log applicatif est distinct du log technique (erreurs, performances).

Risque concret	Sans audit log, impossible de savoir qui a supprimé des données, ni quand une intrusion a commencé.
Exemple de faille	Aucun log sur <code>/admin/users</code> , un accès non autorisé passe inaperçu
À vérifier dans l'app	<code>grep -rn "logger\" vuln-app/routes/</code> , vérifier couverture des actions sensibles
Correction	Ajouter un log sur chaque action sensible avec <code>userId</code> , <code>IP</code> , <code>timestamp</code>
Test de validation	Effectuer un login, une création de tâche, une action admin → vérifier dans <code>logs/app.log</code>
Trace rapport	« Un audit log applicatif couvre les événements : login/logout, création/suppression de tâche, accès admin. Chaque entrée inclut <code>userId</code> , <code>IP</code> , <code>timestamp</code> et résultat. »

Événements à logger (liste minimale) :

```
// Routes à couvrir avec des logs d'audit
const AUDIT_EVENTS = {
  USER_REGISTER: 'user_register',
  USER_LOGIN: 'user_login',
  USER_LOGOUT: 'user_logout',
  USER_LOGIN_FAILED: 'user_login_failed',
  TASK_CREATE: 'task_create',
  TASK_DELETE: 'task_delete',
  ADMIN_ACCESS: 'admin_access',
  ADMIN_USER_DELETE: 'admin_user_delete',
  PASSWORD_CHANGE: 'password_change',
};

// Fonction d'audit log centralisée
function auditLog(action, req, data = {}) {
  logger.info(action, {
    userId: req.session?.userId || 'anonymous',
    ip: req.ip,
    userAgent: req.headers['user-agent'],
    timestamp: new Date().toISOString(),
    ... data,
  });
}

// Usage dans les routes
app.delete('/tasks/:id', requireAuth, (req, res) => {
  auditLog(AUDIT_EVENTS.TASK_DELETE, req, { taskId: req.params.id });
  // ... logique de suppression
});
```

CONCEPT 8, Rapport de Sécurisation Professionnel

Rappel. Le rapport de sécurisation est le livrable principal d'une mission de sécurisation. Il doit être compréhensible par un décideur technique (CTO, DSI) et par les développeurs qui feront la remédiation. Il suit une structure standard : contexte et périmètre, méthodologie utilisée, vulnérabilités trouvées classées par sévérité, corrections appliquées avec preuves, tests de validation, risques résiduels, et recommandations. Chaque vulnérabilité reçoit une fiche avec : description, CVSS estimé, preuve (capture ou log), correction appliquée, statut.

Risque concret	Un rapport incomplet ou non structuré est inutilisable pour prioriser les corrections ou justifier un investissement sécurité.
Exemple de faille	Rapport sans preuve → vulnérabilité contestée par l'équipe dev
À vérifier dans l'app	Collecter toutes les captures dans <code>proofs/M*/</code> avant de rédiger
Correction	Utiliser le template ci-dessous
Test de validation	Rapport relu par un pair, toutes les sections remplies
Trace rapport	(Cette section est le rapport lui-même)

Voir le Modèle de Rapport complet en section dédiée ci-dessous.

CONCEPT 9, Checklist OWASP Top 10

Rappel. L'OWASP Top 10 est la liste des 10 catégories de risques web les plus critiques, mise à jour tous les 3-4 ans à partir de données réelles (CVE, enquêtes, contributions de la communauté sécurité). Elle est un standard de référence pour les audits de conformité. Cocher la checklist permet de s'assurer qu'aucune catégorie majeure n'a été oubliée.

Risque concret	Se concentrer sur SQLi (A03) et oublier A01 (Broken Access Control), la vulnérabilité la plus répandue selon OWASP 2021.
Exemple de faille	TaskFlow initial : A01 (IDOR), A02 (mots de passe MD5), A03 (SQLi), A05 (pas de Helmet)
À vérifier dans l'app	Remplir la checklist en section dédiée
Correction	Appliquer les corrections des modules M1-M7
Test de validation	Checklist complétée avec preuves pour chaque item
Trace rapport	« La checklist OWASP Top 10 2021 a été complétée. 8 catégories sur 10 sont adressées. A07 et A10 sont en risques résiduels (roadmap Q2). »

CONCEPT 10, Risques Résiduels et Plan d'Amélioration

Rappel. Un audit de sécurisation n'élimine pas tous les risques, il les réduit à un niveau acceptable. Les risques résiduels sont ceux qui subsistent après corrections, soit parce que la correction est trop coûteuse, soit parce qu'elle dépend d'une décision architecturale. Chaque risque résiduel doit être documenté avec : description, impact potentiel, probabilité estimée, décision (accepter/transférer/réduire), et responsable. Le plan d'amélioration priorise les corrections futures.

Risque concret	Un risque non documenté peut être oublié lors d'une prochaine revue et se transformer en incident.
Exemple de faille	Rate limiting absent sur <code>/login</code> , brute-force possible → risque résiduel si non corrigé en M4
À vérifier dans l'app	Lister tous les items non couverts par les modules M1-M7
Correction	Documenter dans la section "Risques résiduels" du rapport
Test de validation	Chaque risque résiduel a un responsable et une date de traitement estimée
Trace rapport	« 3 risques résiduels identifiés : [liste]. Plan de remédiation Q2 2024 inclus dans le rapport. »

TP Complet M7 (90 min)

Étape 1, Installer OWASP ZAP + Scan baseline (20 min)

```
# Option Docker (recommandée)
docker pull ghcr.io/zaproxy/zaproxy:stable

# Démarrer l'application
cd vuln-app && node app.js &

# Lancer le scan
mkdir -p ../proofs/M7
docker run --rm --network host \
  -v "$(pwd)/../proofs/M7:/zap/wrk" \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py \
  -t https://localhost:3443 \
  -r zap-report.html \
  -J zap-report.json \
  -I

echo "Rapport dans proofs/M7/zap-report.html"
```

Analyser le rapport :

```
# Compter les alertes par sévérité (nécessite jq)
cat proofs/M7/zap-report.json | \
  jq ' [.site[] .alerts[] | {risk: .riskdesc, name: .name}] | group_by(.risk)'
```

✓ **Point de vérification** : rapport HTML généré, alertes listées par sévérité.

Étape 2, npm audit + remédiation (15 min)

```
cd vuln-app

# Audit avant corrections
npm audit --json > ../proofs/M7/npm-audit-before.json
npm audit # lire le résumé

# Corrections automatiques
npm audit fix

# Vérification
npm audit
npm audit --json > ../proofs/M7/npm-audit-after.json

# Si des vulnérabilités persistent (acceptées ou nécessitant --force)
# Documenter dans le rapport
```

✓ **Point de vérification** : 0 vulnérabilité HIGH ou CRITICAL restante.

Étape 3, Écrire 5 tests Jest de sécurité (25 min)

```
cd vuln-app
npm install --save-dev jest supertest
mkdir -p tests/security

# Copier le fichier security.test.js (Concept 6)
# Adapter les endpoints selon l'état réel de l'application

npm test -- tests/security/ --verbose

# Sauvegarder le résultat
npm test -- tests/security/ --verbose 2>&1 | tee ../proofs/M7/jest-results.txt
```

✓ **Point de vérification** : 5 passed, 0 failed .

Étape 4, Remplir le rapport (20 min)

Utiliser le template de rapport ci-dessous. Remplir chaque section avec les données collectées lors des modules M1-M7.

```
mkdir -p docs
# Créer docs/rapport-final.md à partir du template
# Exporter en PDF via Pandoc ou impression navigateur
```

Étape 5, Remplir la checklist OWASP (5 min)

Remplir la checklist de la section dédiée ci-dessous.

Étape 6, Risques résiduels (5 min)

Remplir la section “Risques résiduels” du rapport avec les items non traités.

Modèle de Rapport de Sécurisation Professionnel

```
# Rapport de Sécurisation, TaskFlow API
**Version :** 1.0
**Date :** [DATE]
**Rédigé par :** [NOM ÉTUDIANT]
**Période d'audit :** [DATE DÉBUT], [DATE FIN]

---

## 1. Contexte et Périmètre

### 1.1 Présentation de l'application
TaskFlow est une API REST Node.js/Express/SQLite exposant les fonctionnalités :
- Gestion de compte utilisateur (register, login)
- Gestion de tâches (CRUD)
- Interface d'administration (`/admin/users`)

**Stack technique :** Node.js [VERSION], Express [VERSION], SQLite3, JWT
**Environnement testé :** Développement local (localhost:3443, mkcert)
**Périmètre :** Code source `vuln-app/`, base de données SQLite, configuration serveur

### 1.2 Objectifs de la mission
Identifier et corriger les vulnérabilités de sécurité dans le code source,
durcir la configuration HTTP, et documenter les risques résiduels.

---

## 2. Méthodologie

| Phase | Outil / Méthode | Module |
|-----|-----|-----|
| Revue de code manuelle | Lecture + grep | M1-M6 |
| Tests d'injection | Payloads manuels + sqlmap | M2-M3 |
| Analyse dynamique (DAST) | OWASP ZAP baseline | M7 |
| Analyse des dépendances (SCA) | npm audit | M7 |
| Tests automatisés | Jest + supertest | M7 |
| Référentiel | OWASP Top 10 2021 | Tous |

---

## 3. Vulnérabilités Identifiées

### 3.1 Résumé exécutif

| Sévérité | Nombre | Corrigées | Résiduelles |
|-----|-----|-----|-----|
| Critique | X | X | 0 |
| Haute | X | X | X |
```

```
| Moyenne | X | X | X |
| Faible  | X | X | X |
| **Total** | **X** | **X** | **X** |
```

3.2 Fiches de vulnérabilité

VULN-01, Injection SQL (Haute)

****OWASP**** : A03:2021, Injection

****CVSS estimé**** : 8.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)

****Description****

La route `POST /login` construisait la requête SQL par concaténation directe de l'entrée utilisateur, permettant à un attaquant de contourner l'authentification.

****Preuve (avant correction)****

```
```js
```

```
// auth.js ligne 23, extrait du code original
```

```
const query = `SELECT * FROM users WHERE email='${email}'`;
```

## Exploitation démontrée

```
curl -X POST http://localhost:3000/login \
 -d "email=' OR 1=1--&password=anything"
Résultat : connexion réussie sans authentification
```

## Correction appliquée

```
// auth.js ligne 23, après correction
db.get('SELECT * FROM users WHERE email = ?', [email], ...)
```

**Statut** : ☒ Corrigé, commit [HASH]

**Test de validation** : `POST /login` avec payload `' OR 1=1--` → 401 Unauthorized

## VULN-02, MOTS DE PASSE STOCKÉS EN CLAIR (CRITIQUE)

**OWASP** : A02:2021, Cryptographic Failures

**CVSS estimé** : 9.1

### Description

Les mots de passe étaient stockés en clair dans la base de données SQLite.

### Preuve (avant correction)

```
sqlite3 taskflow.sqlite "SELECT email, password FROM users;"
admin@test.com | password123
```

### Correction appliquée

Intégration de `bcrypt` avec facteur de coût 12.

**Statut :** ☒ Corrigé, commit [HASH]

## VULN-03, IDOR SUR LES TÂCHES (HAUTE)

**OWASP :** A01:2021, Broken Access Control

**CVSS estimé :** 7.5

### Description

Un utilisateur authentifié pouvait accéder aux tâches de n'importe quel autre utilisateur en incrémentant l'ID dans l'URL.

### Exploitation démontrée

```
curl http://localhost:3000/tasks/1 -H "Cookie: connect.sid=SESSION_USER2"
Retournait les tâches de l'utilisateur 1, fuite de données
```

### Correction appliquée

Ajout de `WHERE id = ? AND userId = ?` dans toutes les requêtes de tâches.

**Statut :** ☒ Corrigé, commit [HASH]

## VULN-04, ABSENCE DE HEADERS DE SÉCURITÉ (MOYENNE)

**OWASP :** A05:2021, Security Misconfiguration

### Description

Aucun header HTTP de sécurité (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) n'était présent.

**Correction appliquée :** Helm.js configuré (M5).

**Statut :** ☒ Corrigé, commit [HASH]

## VULN-05, SECRETS CODÉS EN DUR (HAUTE)

**OWASP :** A02:2021, Cryptographic Failures

### Description

`JWT_SECRET = "secret123"` présent dans `auth.js`.

**Correction appliquée :** Migration vers `.env` + `openssl rand -base64 64` (M6).

**Statut :** ☒ Corrigé, commit [HASH]

---

## VULN-06, PAS DE CSRF (HAUTE)

**OWASP :** A01:2021, Broken Access Control

### Description

Routes POST acceptant des requêtes cross-site sans vérification de token.

**Correction appliquée :** `csrf` middleware (M5).

**Statut :** ☒ Corrigé, commit [HASH]

---

## VULN-07, LOGS EXPOSANT DES DONNÉES SENSIBLES (MOYENNE)

**OWASP :** A09:2021, Security Logging and Monitoring Failures

### Description

`console.log("login:", email, password)` , mot de passe en clair dans les logs.

**Correction appliquée :** Winston + audit des `console.log` (M6).

**Statut :** ☒ Corrigé, commit [HASH]

---



## 4. Tests de Validation

TEST	MÉTHODE	RÉSULTAT ATTENDU	STATUT
SQLi login	' OR 1=1 -- en email	401 Unauthorized	✓ Passe
IDOR tâches	GET /tasks/1 user2	403 Forbidden	✓ Passe
XSS stocké	<script> dans titre tâche	Payload échappé	✓ Passe
Brute-force	10 logins échoués/min	429 Too Many Requests	✓ Passe
Headers sécurité	curl -I https://localhost:3443	CSP + HSTS + XFO présents	✓ Passe
CSRF	POST sans token	403 invalid csrf token	✓ Passe
Stack trace	Route en erreur	Message générique	✓ Passe
Mots de passe	SELECT password FROM users	\$2b\$12\$ ...	✓ Passe

## 5. Résultats OWASP ZAP

**Date du scan :** [DATE]

**Version ZAP :** [VERSION]

**URL cible :** https://localhost:3443

**Mode :** Baseline Scan

SÉVÉRITÉ	AVANT CORRECTION	APRÈS CORRECTION
High	X	0
Medium	X	X
Low	X	X
Informational	X	X

Rapport complet : [proofs/M7/zap-report.html](#)

## 6. Risques Résiduels

#	VULNÉRABILITÉ	OWASP	SÉVÉRITÉ	RAISON DE NON-CORRECTION	PLAN
R1	Rate limiting login	A07	Moyenne	Hors périmètre M1-M7	Sprint suivant
R2	Pas de 2FA	A07	Faible	Complexité fonctionnelle	Roadmap Q2
R3	Logs non centralisés	A09	Faible	Infrastructure non disponible	Prod uniquement

## 7. Conclusion

[NOM ÉTUDIANT] a réalisé l'audit de sécurisation de TaskFlow sur la période [DATES]. Les 7 vulnérabilités critiques et hautes identifiées ont été corrigées. 3 risques résiduels de sévérité moyenne à faible ont été documentés avec un plan de remédiation.

La posture de sécurité de l'application a significativement progressé : - Score sécurité estimé avant : 2/10 - Score sécurité estimé après : 7.5/10

**Livrables joints :** - Code source taggé `M7-audit-final` - Rapport ZAP : `proofs/M7/zap-report.html` - Tests Jest : `tests/security/security.test.js` - npm audit : `proofs/M7/npm-audit-after.json` ``

## Checklist OWASP Top 10, TaskFlow

Cocher chaque item après validation :

#	CATÉGORIE OWASP	FAILLE INITIALE	MODULE	STATUT
A01	Broken Access Control	IDOR sur /tasks, /admin sans auth	M3 · M4	<input type="checkbox"/>
A02	Cryptographic Failures	Mots de passe MD5/clair, JWT secret faible	M2 · M6	<input type="checkbox"/>
A03	Injection	SQLi sur /login et /tasks	M2	<input type="checkbox"/>
A04	Insecure Design	Pas de rate limiting, énumération users	M4	<input type="checkbox"/>
A05	Security Misconfiguration	Pas de headers Helmet, HTTP en clair	M5	<input type="checkbox"/>
A06	Vulnerable Components	Dépendances non auditées	M7	<input type="checkbox"/>
A07	Auth & Session	Session non expirée, pas de logout sécurisé	M4	<input type="checkbox"/>
A08	Software Integrity	package-lock.json non commité	M7	<input type="checkbox"/>
A09	Logging Failures	Logs avec mots de passe, pas de Winston	M6	<input type="checkbox"/>
A10	SSRF	Non applicable (pas de requête vers URL externe)	,	<input type="checkbox"/> N/A

\*\*Score final : \_\_\_\_/9 catégories adressées\*\*

## Mini-quiz M7, Synthèse du cours (10 QCM)

**Q1.** La différence principale entre SAST et DAST est : - A. SAST analyse le code statique, DAST teste l'application en exécution - B. SAST est automatique, DAST est manuel - C. SAST couvre toutes les vulnérabilités, DAST seulement les headers - D. DAST nécessite l'accès au code source

**Q2.** `npm audit fix --force` : - A. Supprime toutes les vulnérabilités garanties - B. Applique des mises à jour majeures qui peuvent casser l'API - C. Est équivalent à `npm audit fix` mais plus rapide - D. Ignore les faux positifs

**Q3.** Quel type de vulnérabilité `npm audit` détecte-t-il ? - A. Injections SQL dans le code - B. Vulnérabilités des dépendances tierces - C. Headers HTTP manquants - D. Fuites de secrets dans le code

- Q4.** Dans un test Jest de sécurité, si le test “SQLi retourne 401” **échoue**, cela signifie : - A. Le test est mal écrit - B. L’injection SQL fonctionne, la protection est absente ou a régressé - C. SQLite n’est pas démarré - D. Jest n’est pas configuré pour les tests de sécurité
- Q5.** `bcrypt.hash(password, 12)`, que représente `12` ? - A. La longueur du sel - B. Le facteur de coût : plus élevé = calcul plus lent = plus résistant au brute-force - C. Le nombre de rounds SHA - D. La taille minimale du mot de passe
- Q6.** Dans un rapport de sécurité, un “risque résiduel” est : - A. Une vulnérabilité non découverte - B. Une vulnérabilité connue, documentée, et acceptée ou reportée - C. Une erreur dans le rapport - D. Un faux positif ZAP
- Q7.** `X-Content-Type-Options: nosniff` protège contre : - A. Le clickjacking - B. Le MIME type sniffing, le navigateur exécute un fichier texte comme JavaScript - C. Les injections XSS directes - D. Les requêtes CSRF
- Q8.** `git log -S "secret123"` permet de : - A. Supprimer les commits contenant “secret123” - B. Rechercher dans l’historique Git tous les commits ayant ajouté ou supprimé “secret123” - C. Chiffrer les commits contenant ce mot - D. Vérifier le message de commit
- Q9.** La directive CSP `script-src 'self'` signifie : - A. Seuls les scripts inline sont autorisés - B. Seuls les scripts provenant de la même origine sont exécutés - C. Tous les scripts externes sont autorisés - D. Les scripts sont désactivés
- Q10.** Le CVSS (Common Vulnerability Scoring System) est : - A. Un outil de scan automatique - B. Un système de notation standardisé de la sévérité des vulnérabilités (0-10) - C. La liste des vulnérabilités OWASP - D. Un format de rapport de sécurité

## Corrigé

Q	RÉPONSE	JUSTIFICATION
Q1	A	DAST = blackbox runtime, SAST = analyse du code source
Q2	B	<code>--force</code> applique des breaking changes, à vérifier manuellement
Q3	B	<code>npm audit</code> compare les dépendances à la base CVE du registre npm
Q4	B	Un test de sécurité qui échoue = la faille est présente
Q5	B	Le facteur de coût contrôle le temps de calcul, 12 = ~0.3s sur CPU moderne
Q6	B	Risque accepté et documenté, pas ignoré
Q7	B	Sans ce header, un navigateur peut “deviner” le type MIME et exécuter du JS
Q8	B	<code>-S</code> (pickaxe) recherche dans l'historique des diffs, utile pour audit secrets
Q9	B	<code>'self'</code> = même origine : même schéma + hôte + port
Q10	B	CVSS donne un score 0-10 : base, temporel, environnemental

## Livrable Final du Module

- ☐ Code taggé : `git tag M7-audit-final && git push --tags`
- ☐ Rapport `docs/rapport-final.md` rédigé (toutes sections complètes)
- ☐ Rapport PDF généré : `docs/rapport-final.pdf`
- ☐ Checklist OWASP complétée
- ☐ Tests Jest dans `tests/security/security.test.js` , 5 passed
- ☐ Capture scan ZAP dans `proofs/M7/zap-report.html`
- ☐ npm audit after dans `proofs/M7/npm-audit-after.json`

## Erreurs fréquentes à éviter

- Lancer ZAP sur une application arrêtée → 0 résultats, scan inutile.
- Accepter tous les findings ZAP sans les analyser → les faux positifs gonflent le rapport et perdent la confiance du lecteur.
- Écrire des tests Jest qui passent toujours (assertions trop permissives) → ils ne détectent aucune régression.
- Rapport sans preuves → vulnérabilités contestables.
- Oublier de commiter `package-lock.json` → builds non reproductibles.
- Marquer tous les risques résiduels comme “acceptés” sans justification → le rapport perd sa valeur.

## Ressources

- OWASP Top 10 2021 : <https://owasp.org/www-project-top-ten/>
- OWASP Testing Guide (OTG) : <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP ZAP : <https://www.zaproxy.org/>
- OWASP ZAP Docker : <https://www.zaproxy.org/docs/docker/about/>
- npm audit documentation : <https://docs.npmjs.com/cli/v10/commands/npm-audit>
- Jest documentation : <https://jestjs.io/docs/getting-started>
- supertest : <https://github.com/ladjs/supertest>
- CVSS Calculator : <https://www.first.org/cvss/calculator/3.1>
- truffleHog (scan historique Git) : <https://github.com/trufflesecurity/trufflehog>
- BFG Repo Cleaner (supprimer secrets de l'historique) : <https://rtyley.github.io/bfg-repo-cleaner/>